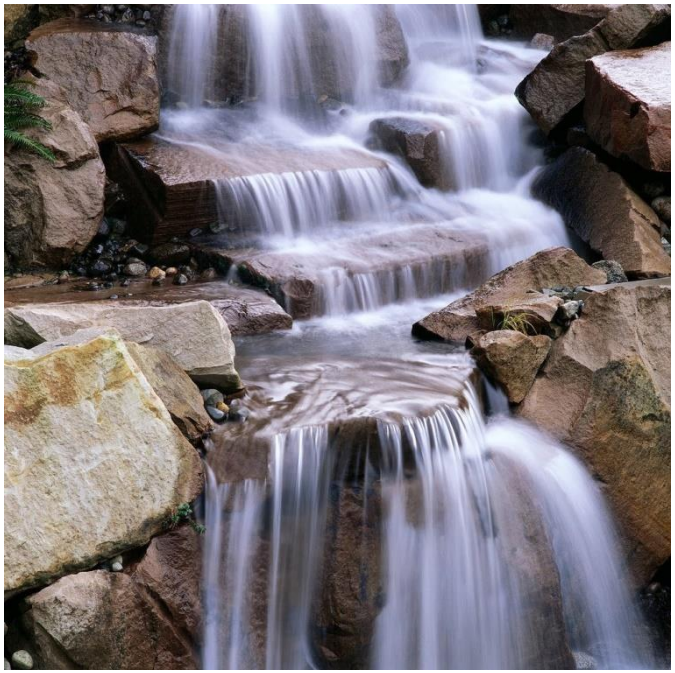




Maximum Flow – part 2



Reading:

CLRS

chapter 27 (1st Ed.)
chapter 26 (2nd and 3rd Ed.)

Review – Lecture 10

Max Flow:

Input: a directed graph (network) G.

a capacity function $c: E \to \mathbb{R}^{\geq 0}$

a specified source node s and target node t.

Output: a flow function $f: E \to \mathbb{R}^{\geq 0}$.

f(e) is the amount of flow on edge e.

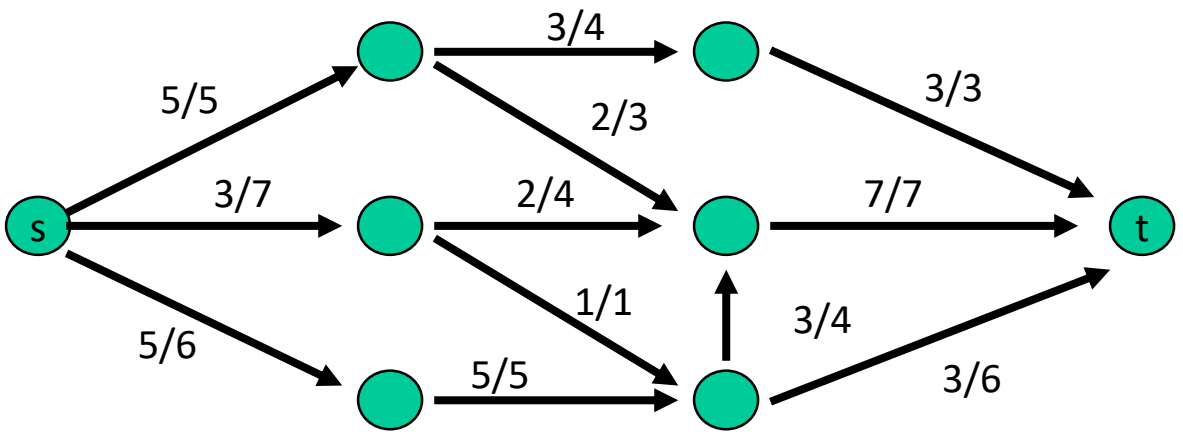
Legal flow:

Edge condition: for all e, $0 \leq f(e) \leq c(e)$.

Vertex condition: for all v except s,t: $\sum_{e \in in(v)} f(e) = \sum_{e \in out(v)} f(e)$.

Flow value:

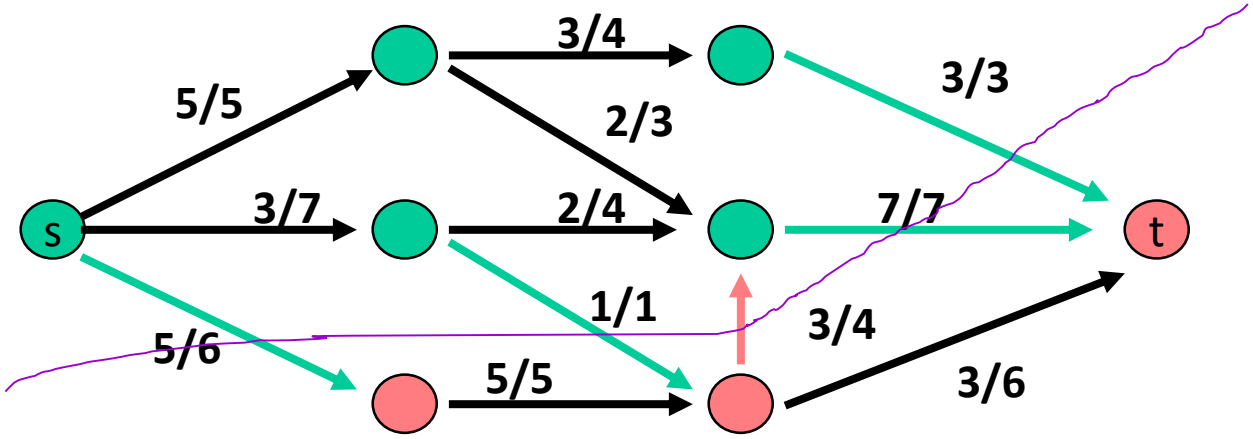
$F = \sum_{e \in out(s)} f(e)$.



Review – Lecture 10

An **s-t cut** (S, V-S) is defined by a set of vertices, S,
Such that $s \in S$, $t \notin S$, and s is a root of the graph induced by S.

out(S): the set of edges (u,v) such that $u \in S$ and $v \notin S$
in(S): the set of edges (u,v) such that $v \in S$ and $u \notin S$



Review – Lecture 10

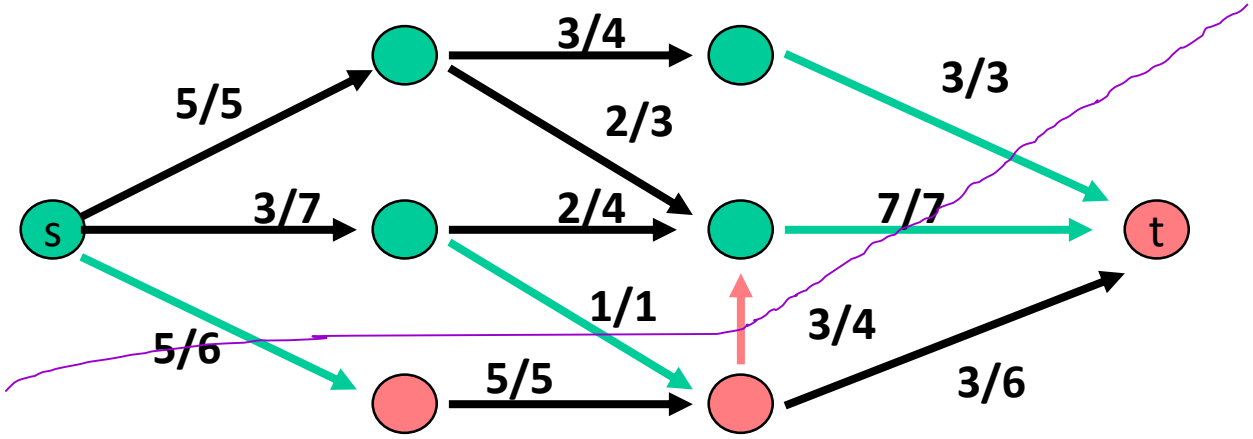
Claim: for any cut S and legal flow f: $F = \sum_{e \in out(S)} f(e) - \sum_{e \in in(S)} f(e)$.

Capacity of a cut = $\sum_{e \in out(S)} c(e)$.

In the example: F=13

C(S)=17

13=16-3



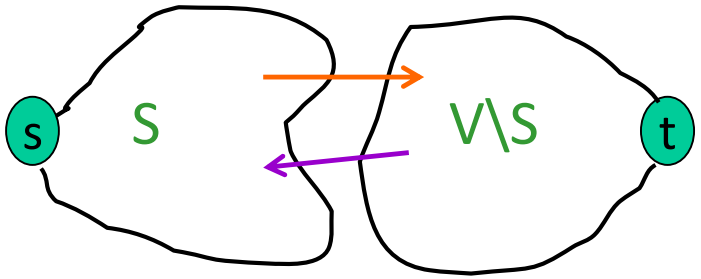
Review – Lecture 10

Claim:

For every legal flow and every cut (S, V-S), $F \leq c(S)$

The Max-flow Min-Cut Theorem:

The value of a maximum flow in a network is equal to the minimum capacity of an (s-t) cut.

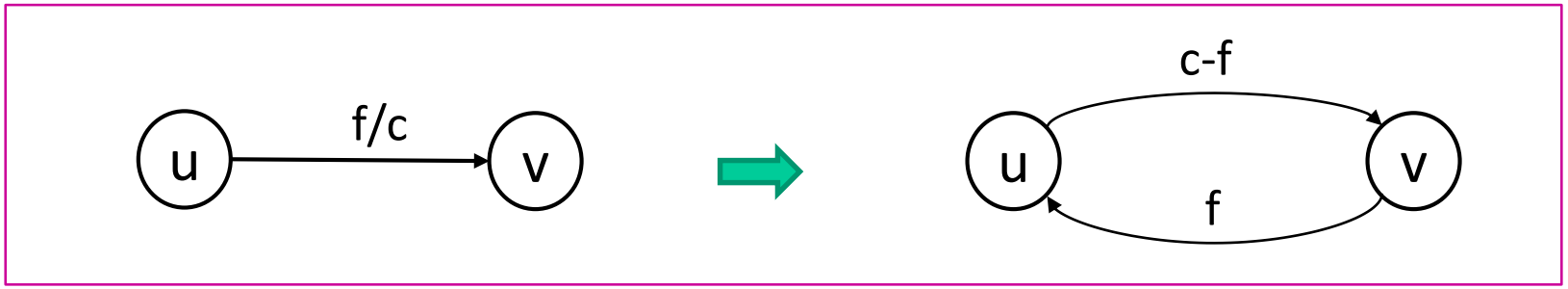


Review – Lecture 10

Finding a max-flow: F&F method.

- Initialize flow on all edges to 0.
- While there is an augmenting path, improve the flow along this path.

Implementation: In every iteration, given G,c,f, we build the residual graph R_f.



An augmenting path is a path from s to t in R_f

Review – Lecture 10

In every iteration of F&F:

1. Build R_f
2. Find a path p from s to t in R_f.
3. δ = minimum capacity along p in R_f.
4. Augment δ units of flow along p and update f.

Until: No path from s to t in R_f.

Correctness:

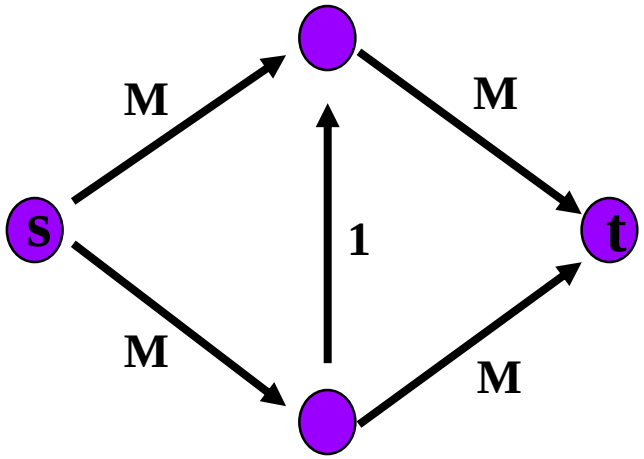
1. The flow is valid.
2. Some cut is saturated: Let S be the set of nodes reachable from S in the last residual network, then for all edges in OUT(S): f(e) = c(e), and for all edges in IN(S): f(e) = 0.
3. This implies that F=c(S), and we know we can't do better.

Running Time of Ford-Fulkerson

Each iteration (building R_f and detecting an augmenting path) takes $O(|E|)$ (how?).

How many iterations are there?

If all capacities are integers, then in every iteration, $\delta \geq 1$. The number of iterations is at most f^*, when f^* is the value of the maximum flow.





The time complexity of FF is $O(f^*|E|)$, when f^* is the value of the maximum flow.

Running Time of Ford-Fulkerson

- Can we bound the number of iterations if the capacities are not integers?

  Unfortunately, no. There exists a flow network with real capacities such that Ford-Fulkerson does not terminate.

  Moreover, the value of the flow converges to a value arbitrarily far from the max flow.

- Can we bound the number of iteration as a function of $|V|$ and $|E|$? 

- Yes – both issues can be solved by a wise selection of augmenting paths. 

Edmonds-Karp Algorithm:

Use FF method. Search for augmenting path using breadth-first search, i.e., the augmenting path is a **shortest path** (minimal number of edges) from s to t in the residual network.

Theorem (proof omitted): This way, the number of augmentations is $O(|E||V|)$

The resulting complexity: $O(|E|^2|V|)$

(each iteration takes $O(|E|)$)

Greedy augmenting path Selection

Use FF method. In each iteration select an augmenting path with the maximal δ value.

The time complexity of this algorithm is

$O(|E|^2 \log |E| \log f^*)$.

Explanation (proof omitted): The widest augmenting path can be computed in time $O(|E| \log |E|)$, and there will be at most $O(|E| \log f^*)$ iterations.

Summing up:

- Edmonds-Karp (EK): $O(|E|^2|V|)$
- If the capacities are all integers, then any application of FF (in particular EK) takes $O(f^*|E|)$
- There are better algorithms.
  - Push-Relabel (Goldberg–Tarjan) : $O(|V|^3)$
  - Dinic: $O(|V|^2|E|)$
  - Capacity Scaling: $O(|E|^2 \log U)$, where U is the max capacity
- In some cases, there is a tighter worst-case analysis of these algorithms for specific classes of graphs (planar, bipartites, etc.)

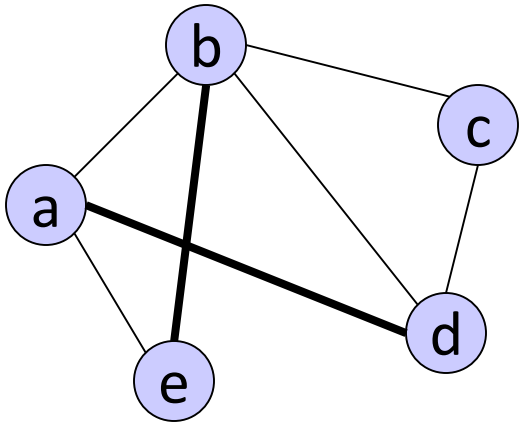
Some applications of *max-flow* and the *max-flow min-cut* theorem

- Network routing, bandwidth allocation
- Bipartite matching
- Assignments with constraints
- Network connectivity
- Image segmentation and computer vision
- Sports Elimination
- Social network analysis
- Many many more…

Matching

- Definition: a matching in a graph G=(V,E) is a subset M of E such that the degree of each vertex in G'=(V,M) is 0 or 1.
- Example: M={(a,d),(b,e)} is a matching.

  S={(a,d), (c,d)} is not a matching.



Bipartite Matching

Example 1: In a party of a heterosexual community there are n_1 boys and n_2 girls. Each boy tells the DJ the girls with whom he is ready to dance with. Each girl tells the DJ the boys with whom she is ready to dance with.

- DJ's goal: As many dancing pairs as possible.

Example 2 (production planning): n_2 identical servers need to serve n_1 clients. For each client, we are given the subset of servers that can serve it.

- Goal: Serve as many clients as possible.

Bipartite Matching

Graph representation: G = (V,E).

$V = V_1 \cup V_2$.

$|V_1| = n_1$ (boys, clients), $|V_2| = n_2$ (girls, servers).

In 1st problem $(u,v) \in E$, if u is ready to dance with v and vice versa.

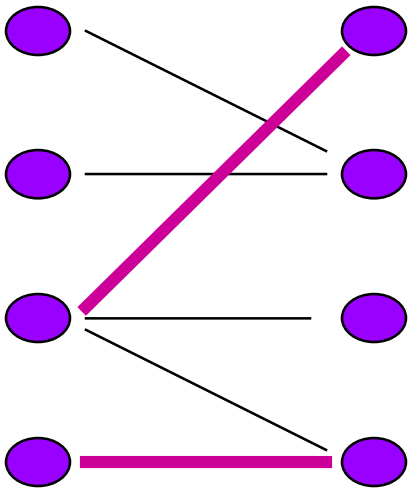
In 2nd problem $(u,v) \in E$, if u can be served by v.

This is a bipartite!

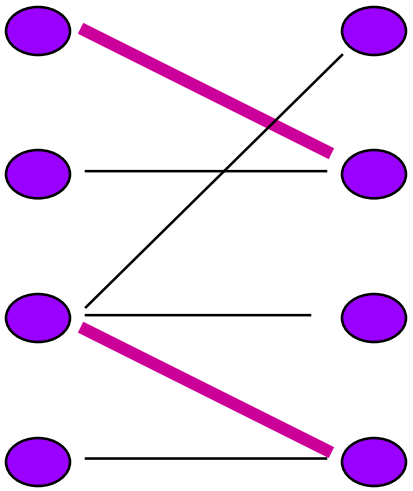
We are looking for the largest possible matching.

Bipartite Matching

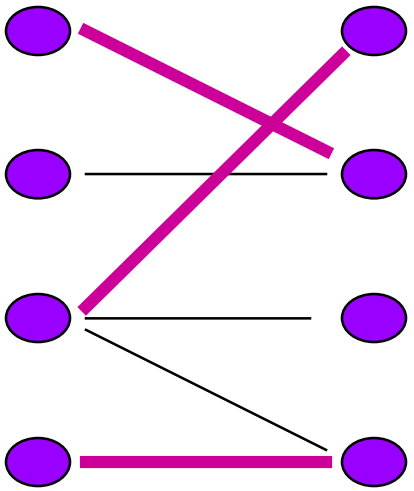
- Input: a bipartite graph $G=(V_1 \cup V_2, E)$
- Goal: A matching of maximal size.



A matching



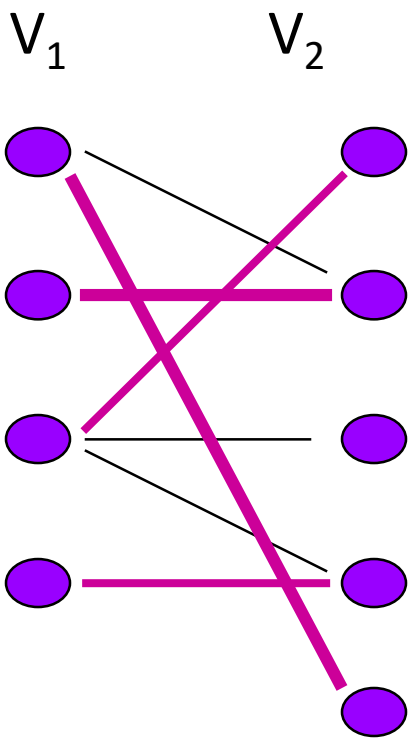
A maxim**al** matching – can not be extended.



A maxim**um** matching – largest maximal size.

Our goal !

Bipartite Matching



Special cases:

- A perfect matching: $|M|=|V_1|=|V_2|$
- A full matching for V_1: $|M|=|V_1|\leq|V_2|$

Maximum matching in a bipartite can be found using flow algorithms.

Using Flow for Bipartite Matching

Input: A bipartite $G = (V_1 \cup V_2, E)$

Output : Maximum matching $M \subseteq E$.

Algorithm:

1. Build a network flow $N = (V', E')$

   $V' = V_1 \cup V_2 \cup \{s,t\}$

   $E' = E \cup \{(s \rightarrow u) \mid \forall u \in V_1\} \cup \{(v \rightarrow t) \mid \forall v \in V_2\}$

   All $e \in E'$ have the capacity $c(e)=1$.

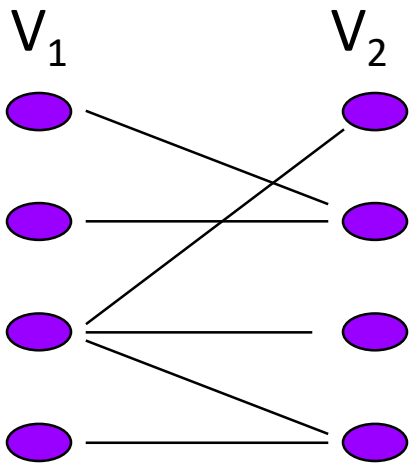
Edges of E are directed from V_1 to V_2

2. Find a maximum flow in N.

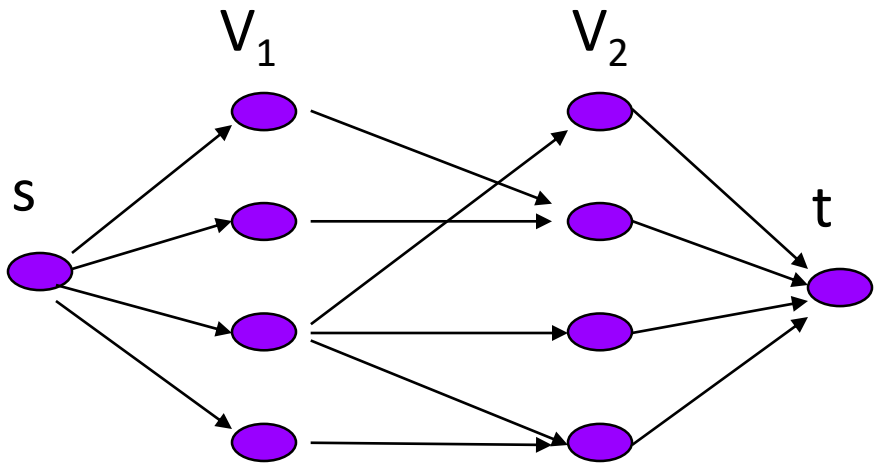
3. M = saturated edges in the cut defined by $\{s\} \cup V_1$
   (Meaning: all saturated edges crossing from V1 to V2)

Using Flow for Bipartite Matching (Example)

$G = (V_1 \cup V_2, E)$



$N = (V', E')$



$V' = V_1 \cup V_2 \cup \{s,t\}$

$E' = E \cup \{(s \rightarrow u) \mid \forall u \in V_1\}$
$\cup \{(v \rightarrow t) \mid \forall v \in V_2\}$

For all $e \in E'$, $c(e) = 1$.

Using Flow for Bipartite Matching (proof)

Theorem: G includes a matching of size k $\Leftrightarrow$
N has flow with value k.

Proof:

1. ($\Rightarrow$) Given a matching of size k, define the flow f(u,v) = 1 for all (u,v) in M, and for all (s,u) and (v,t) such that u or v are matched. For all other edges, f = 0.

- f is legal (proof soon)
- The value of f is k (consider the cut $\{s\} \cup V_1$).

2. ($\Leftarrow$) Similar. Based on the capacities of the edges (s,u), (v,t), and the fact that f is legal.

Using Flow for Bipartite Matching (proof)

Proof that f is a legal flow:

Edge condition: All the capacities were set to be 1. The flow on each edge is either 0 or 1.

Vertex condition: By definition of matching, every vertex in V_1 and V_2 has degree 0 or 1 in M.

- If u is matched, then the flow entering u is 1 and the flow leaving u is 1.
- If u is not matched, then the flow entering u and the flow leaving u is 0.

Using Flow for Bipartite Matching

Time complexity:

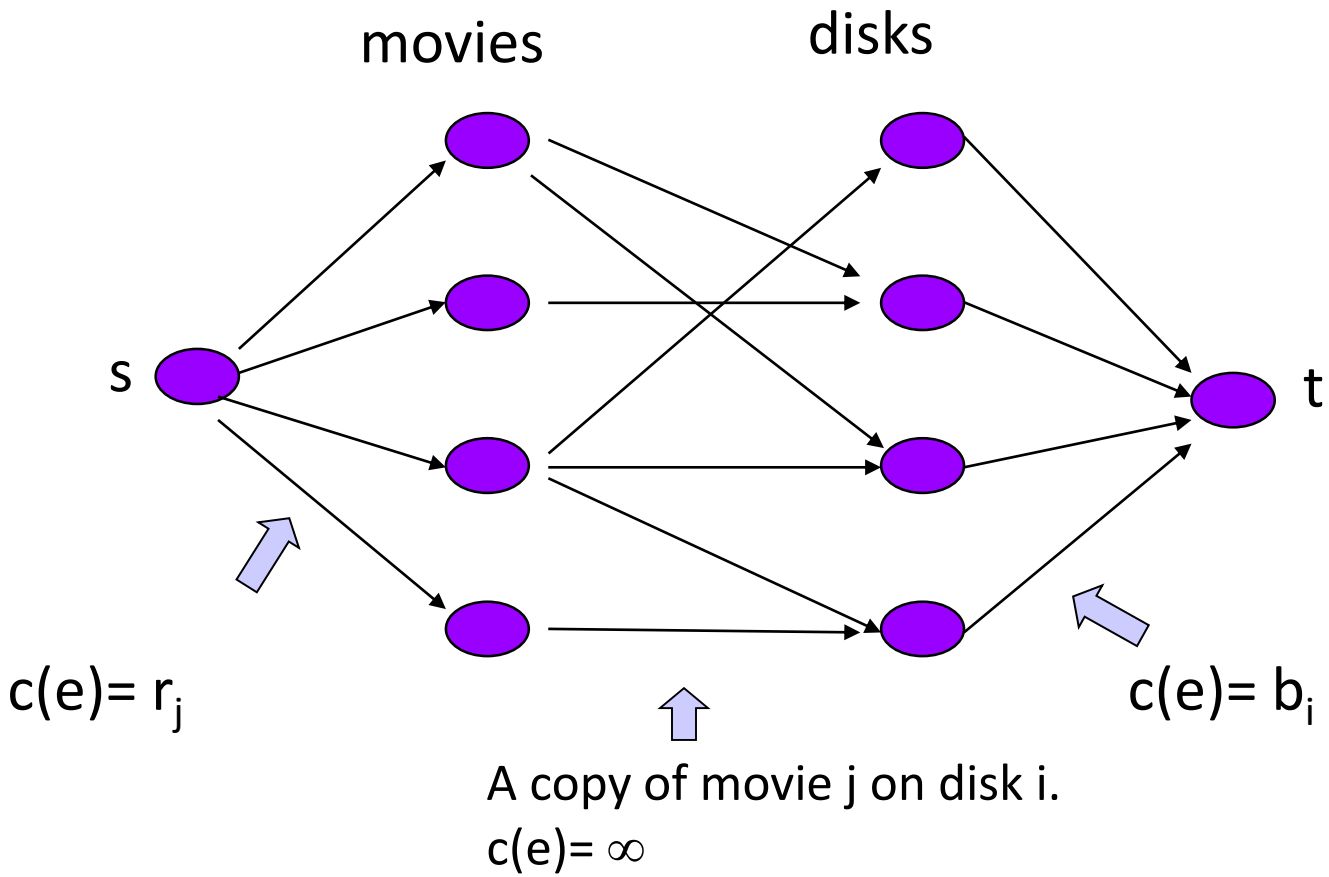
The value of the max-flow is at most $\min(|V_1|,|V_2|) = O(|V|)$.

Since all capacities are integers, any application of F&F algorithm takes $O(|E||V|)$.

Video on Demand

- m storage devices (e.g., disks), The ith disk is capable of supporting b_i simultaneous streams.
- k movies, one copy of each on some of the disks (this assignment is given as input).
- Given set of R movie requests, (r_j requests to movie j) how would you assign the requests to disks so that no disk is assigned more than b_i requests and the maximum number of requests is served?

Video on Demand



Claim: There is a flow of value k if and only if there is an assignment that satisfy k clients.

Other network flow problems:

1. Min-cost max-flow

Input: a graph (network) G where each edge has a capacity c(e), and ***a cost*** cost(e).

Goal: A maximum flow of minimum cost.

- The cost of a flow : $\sum_{f(e)>0} cost(e) \cdot f(e)$

Out of all the maximum flows, which has minimal cost?

2. Multi-commodity flow

Multiple commodities (flow demands) between different source and sink nodes.

The solvability depends on the ability to split the flow on an edge among different commodities.

Other network flow problems

3. Max-flow with lower bounds on flow.

- For each (v,w): $0 \leq lb(v,w) \leq f(v,w) \leq c(v,w)$
- Not always possible:



4. Minimum flow

- send minimum amount of flow from source to sink, while satisfying certain lower and upper bounds on flow on each edge.

Max-Flow
Sample problems.

1. Flow having a specific value
2. Downward critical edges
3. Network Connectivity
4. Vertex-Disjoint paths

Flow having a specific value

Problem: Given a flow network G, and a value $B \geq 0$, suggest an algorithm that returns a flow whose value is exactly B, or announce that such a flow does not exist.

Solution:

1. Given G, build a flow network G' as follows:
   $V' = V \cup \{t'\}$.
   $E' = E \cup \{(t,t')\}$. The new edge has capacity B.
2. Find a max-flow in G' with t' as target.
3. If F=B, return the flow on E,
   else "no flow of value B exists".

Flow having a specific value

Correctness:

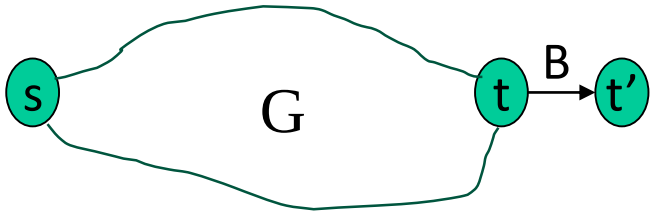
Claim:

The max flow in G' is B $\Leftrightarrow$ G has a flow whose value is B.

Proof:

$\Rightarrow$ The edge (t,t') is saturated, the flow in G is legal and by the vertex condition on node t (in G') its value is B.

$\Leftarrow$ Let f be a valid flow in G whose value is B. Set f(t,t')=B in G'. The vertex condition on note t is fulfilled. f is a max-flow since the cut induced by V'-{t'} is saturated.



Downward Critical Edges.

An edge is downward critical if decreasing the capacity of this edge (even by a small ε) decreases the maximum flow value.
Suggest an algorithm for finding all downward critical edges.

Note the difference: An edge is upward critical if increasing the capacity of this edge (even by a small ε) increases the maximum flow value.

In the recitation: An edge is upward critical if it belongs to all min-cuts.

An edge may be DC but not UC.

Downward Critical Edges.

Observation: e is downward critical iff e belongs to some min-cut.

Proof:

1. if e belongs to some min-cut, then reducing its capacity reduces the min-cut capacity, and therefore also the max-flow value.
2. if e does not belong to a min-cut, then reducing its capacity will not hurt the capacity of a min-cut, and the max-flow value.

Downward Critical Edges.

Wrong attempt 1: Run F&F and return the edges in the min-cut.

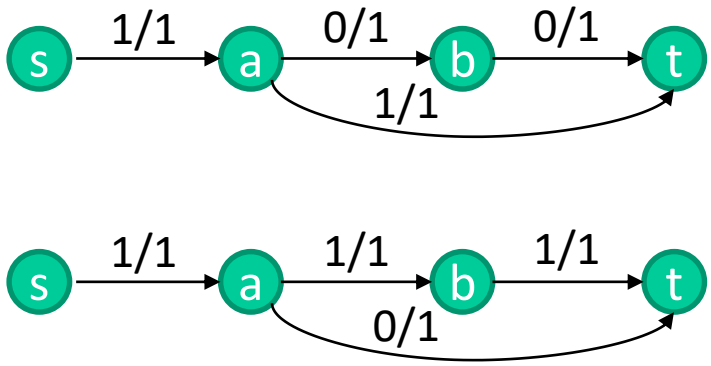
Problem: May return a partial solution.

Example: both edges are d.critical, but only (sa) is in the min-cut detected by F&F.



Downward Critical Edges.

Wrong attempt 2: Run F&F and return all saturated edges.
Problem: May return redundant edges.
Example: In a possible max-flow, both (sa) and (at) are saturated, but only (sa) is d.critical.



Downward Critical Edges.

Algorithm:
Compute the max-flow. Let R_f be the last residual network.
For each saturated edge (u,v) in the original network, run DFS to check whether there is a path from u to v in R_f. If there is a path, (u,v) is not critical, and otherwise it is.

Correctness: Every critical edge has to be saturated (f(e)=c(e)); otherwise, we can reduce the capacity by the slack without affecting the flow.
Assuming e=(u,v) is saturated.
If there is a path from u to v in R_f, we can push flow along the path instead of through e, therefore e is not d.critical.
If there is no path from u to v in R_f, then the flow on it cannot be rerouted in any alternative path and (u,v) is d.critical.

Downward Critical Edges.

Time complexity:

Finding R_f takes $O(nm^2)$,

Running at most m times DFS: $O(m^2)$.

Total: $O(nm^2)$.

Network Connectivity

- Let G be a directed graph representing a communication network. Let u,v be two vertices in G.
- Assume that you want to disable the communication from u to v.
- Suggest an efficient algorithm for detecting a minimum size set of edges, such that if these edges are removed, there is no path from u to v.

Network Connectivity

Solution:

Consider G as a flow network. For all e, set c(e)=1.

Compute a max-flow from u (source) to v (target).

Return the edges of a min-cut.

Time complexity analysis: The value of the flow is bounded by out-degree(u), which is O(|V|).

Therefore, Edmonds-Karp takes O(|E||V|).

Network Connectivity

Correctness

Claim: Let S be a u-v cut. If we remove from the graph the edges of OUT(S) then there is no path from u to v.

Proof: Since $u \in S$ and $v \notin S$, every u-v path includes at least one edge in OUT(S). This implies that at least one edge is removed from any u-v path.

Since we must remove edges of OUT(S) for some u-v cut S, in order to separate v from u, and since all edges have the same unit capacity, a min-cut in the network corresponds to a set of edges of minimal size that should be removed.